PROGRAM 1

Program Code

```
import re

file_path = input("Enter text file path: ").strip()

pattern = r"[A-Za-z0-9+._-]+@[A-Za-z0-9.-]+\.(?:com|edu|org)\b"
data = {}

try:
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            emails = re.findall(pattern, line)
            for email in emails:
                email = email.lower()
                domain = email.split("@")[1]
                if domain not in data:
                    data[domain] = set()
                data[domain].add(email)

    for domain in sorted(data):
        emails = sorted(data[domain])
        print(domain, len(emails), emails[0])
except FileNotFoundError:
    print("File not found")
```

Output

```
Program Output

Enter text file path: adit.edu 1 a.b+l@adit.edu

x.org 1 test-user@x.org
```

PROGRAM 2

Program Code

```
import re

first = input("Enter number of HTML files and K: ").split()
p = int(first[0])
k = int(first[1])

files = []
for i in range(p):
    files.append(input().strip())

products = {}

pattern = re.compile(
    r'<div[^>]*class=["\']product["\'][^>]*>.*?'
    r'<[^>]*class=["\']name["\'][^>]*>(.*?)</[^>]+>.*?'
    r'<[^>]*class=["\']price["\'][^>]*>(.*?)</[^>]+>.*?'
    r'<[^>]*class=["\']rating["\'][^>]*>(.*?)</[^>]+>.*?</div>',
    re.I | re.S
)

try:
    for path in files:
        with open(path, "r", encoding="utf-8") as file:
            html = file.read()

        for name, price, rating in pattern.findall(html):
            name = re.sub(r"<.*?>", "", name).strip()
            price = re.sub(r"[^0-9.]", "", price)
            rating = re.sub(r"[^0-9.]", "", rating)

            if not price or not rating:
                continue

            price = float(price)
            rating = float(rating)

            if name not in products or rating > products[name][1] or (
                rating == products[name][1] and price < products[name][0]
            ):
                products[name] = (price, rating)

    result = list(products.items())
    result.sort(key=lambda x: (-x[1][1], x[1][0], x[0]))

    for name, value in result[:k]:
        price, rating = value
        print(name, int(price) if price.is_integer() else price, rating)
except FileNotFoundError:
    print("HTML file not found")
```

Output

```
Program Output

Enter number of HTML files and K: Python Book 499 4.8

USB Keyboard 799 4.7
```

PROGRAM 3

Program Code

```
import csv

try:
    import mysql.connector
except ImportError:
    print("Install mysql-connector-python first")
    raise SystemExit

host = input("Host: ").strip()
user = input("User: ").strip()
password = input("Password: ").strip()
database = input("Database name: ").strip()

student_file = input("Student CSV path: ").strip()
registration_file = input("Registration CSV path: ").strip()
course_id = input("Course ID: ").strip()
threshold = float(input("SPI threshold: "))

try:
    db = mysql.connector.connect(
        host=host,
        user=user,
        password=password,
        database=database
    )
    cur = db.cursor()

    cur.execute("""
    CREATE TABLE IF NOT EXISTS Student(
        student_id VARCHAR(30) PRIMARY KEY,
        name VARCHAR(100),
        spi FLOAT
    )
    """)

    cur.execute("""
    CREATE TABLE IF NOT EXISTS CourseRegistration(
        student_id VARCHAR(30),
        course_id VARCHAR(30),
        FOREIGN KEY(student_id) REFERENCES Student(student_id)
    )
    """)

    with open(student_file, newline="", encoding="utf-8") as file:
        rows = csv.DictReader(file)
        for row in rows:
            cur.execute(
                "INSERT IGNORE INTO Student(student_id,name,spi) VALUES(%s,%s,%s)",
                (row["student_id"], row["name"], float(row["spi"]))
            )

    with open(registration_file, newline="", encoding="utf-8") as file:
        rows = csv.DictReader(file)
        for row in rows:
            cur.execute(
                "INSERT INTO CourseRegistration(student_id,course_id) VALUES(%s,%s)",
                (row["student_id"], row["course_id"])
            )

    db.commit()

    query = """
    SELECT s.student_id, s.name, s.spi, r.course_id
    FROM Student s
    JOIN CourseRegistration r ON s.student_id = r.student_id
    WHERE r.course_id = %s AND s.spi > %s
    ORDER BY s.spi DESC, s.student_id ASC
    """

    cur.execute(query, (course_id, threshold))

    for row in cur.fetchall():
        print(row[0], row[1], row[2], row[3])

    cur.close()
    db.close()

except Exception as e:
    print("Database error:", e)
```

Output

```
Program Output
2201 Asha 9.1 PY101

2207 Dev 8.4 PY101
```

PROGRAM 4

Program Code

```
import re

allowed_columns = {
    "student.id",
    "student.name",
    "student.spi",
    "course.id",
    "course.name",
    "registration.course_id"
}

columns = input("Columns separated by comma: ").split(",")
columns = [x.strip() for x in columns]

where_text = input("Where condition, example course.id=PY101: ").strip()
order_column = input("Order column: ").strip()
direction = input("Sort direction ASC/DESC: ").strip().upper()
limit = int(input("Limit: "))

if not 1 <= limit <= 1000:
    print("INVALID LIMIT")
    raise SystemExit

if not columns or any(x not in allowed_columns for x in columns):
    print("INVALID COLUMN")
    raise SystemExit

if order_column not in allowed_columns:
    print("INVALID ORDER COLUMN")
    raise SystemExit

if direction not in ["ASC", "DESC"]:
    print("INVALID SORT DIRECTION")
    raise SystemExit

match = re.fullmatch(r"([A-Za-z]+\.[A-Za-z_]+)\s*(=|>|<|>=|<=|LIKE)\s*(.+)", where_text)

if not match:
    print("INVALID WHERE")
    raise SystemExit

where_column = match.group(1)
operator = match.group(2)
value = match.group(3).strip()

if where_column not in allowed_columns:
    print("INVALID WHERE COLUMN")
    raise SystemExit

if value.startswith("'") and value.endswith("'"):
    value = value[1:-1]

if value == "":
    print("INVALID VALUE")
    raise SystemExit

sql = """
SELECT {columns}
FROM Student student
JOIN CourseRegistration registration ON student.id = registration.student_id
JOIN Course course ON registration.course_id = course.id
WHERE {where_column} {operator} %s
ORDER BY {order_column} {direction}
LIMIT %s
""".format(
    columns=", ".join(columns),
    where_column=where_column,
    operator=operator,
    order_column=order_column,
    direction=direction
)

print("SQL_OK")
print(sql.strip())
print("Parameters:", [value, limit])
```

Output

```
Program Output

Columns separated by comma: Where condition, example course.id=PY101: Order column: Sort c
SELECT student.name, course.name

FROM Student student

JOIN CourseRegistration registration ON student.id = registration.student_id

JOIN Course course ON registration.course_id = course.id

WHERE course.id = %s

ORDER BY student.spi DESC

LIMIT %s

Parameters: ['PY101', 2]
```

PROGRAM 5

Program Code

```
import tkinter as tk
from tkinter import messagebox
import re

try:
    import mysql.connector
except ImportError:
    mysql = None

def valid_email(email):
    return re.fullmatch(r"[A-Za-z0-9._+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", email) is not None

def connect_db():
    return mysql.connector.connect(
        host=host.get(),
        user=user.get(),
        password=password.get(),
        database=database.get()
    )

def add_contact():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    phone = phone_entry.get().strip()
    category = category_entry.get().strip()
    notes = notes_entry.get("1.0", tk.END).strip()

    if not name or not valid_email(email):
        messagebox.showerror("Error", "Enter valid name and email")
        return

    try:
        db = connect_db()
        cur = db.cursor()
        cur.execute(
            "INSERT INTO Contact(name,email,phone,category,notes) VALUES(%s,%s,%s,%s,%s)",
            (name, email, phone, category, notes)
        )
        db.commit()
        db.close()
        messagebox.showinfo("Success", "Record inserted")
        show_contacts()
    except Exception as e:
        messagebox.showerror("Database Error", str(e))

def show_contacts():
    listbox.delete(0, tk.END)
    try:
        db = connect_db()
        cur = db.cursor()
        cur.execute("SELECT id,name,email,phone,category FROM Contact ORDER BY name")
        for row in cur.fetchall():
            listbox.insert(tk.END, str(row))
        db.close()
    except Exception as e:
        messagebox.showerror("Database Error", str(e))

def create_table():
    try:
        db = connect_db()
        cur = db.cursor()
        cur.execute("""
        CREATE TABLE IF NOT EXISTS Contact(
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100),
            email VARCHAR(150) UNIQUE,
            phone VARCHAR(30),
            category VARCHAR(50),
            notes TEXT
        )
        """)
        db.commit()
        db.close()
        messagebox.showinfo("Success", "Table is ready")
    except Exception as e:
        messagebox.showerror("Database Error", str(e))

root = tk.Tk()
root.title("Contact Manager")
root.geometry("700x600")

tk.Label(root, text="Host").pack()
host = tk.Entry(root)
host.pack()

tk.Label(root, text="User").pack()
user = tk.Entry(root)
user.pack()
```

```
tk.Label(root, text="Password").pack()
password = tk.Entry(root, show="*")
password.pack()

tk.Label(root, text="Database").pack()
database = tk.Entry(root)
database.pack()

tk.Label(root, text="Name").pack()
name_entry = tk.Entry(root)
name_entry.pack()

tk.Label(root, text="Email").pack()
email_entry = tk.Entry(root)
email_entry.pack()

tk.Label(root, text="Phone").pack()
phone_entry = tk.Entry(root)
phone_entry.pack()

tk.Label(root, text="Category").pack()
category_entry = tk.Entry(root)
category_entry.pack()

tk.Label(root, text="Notes").pack()
notes_entry = tk.Text(root, height=4, width=40)
notes_entry.pack()

tk.Button(root, text="Create Table", command=create_table).pack(pady=5)
tk.Button(root, text="Add Contact", command=add_contact).pack(pady=5)
tk.Button(root, text="Show Contacts", command=show_contacts).pack(pady=5)

listbox = tk.Listbox(root, width=90, height=12)
listbox.pack(pady=10)

root.mainloop()
```

Output

```
Program Output


Record inserted and visible in contact list.
```

PROGRAM 6

Program Code

```
import re
from collections import defaultdict, deque

x, t = map(int, input().split())
n = int(input())

failed = defaultdict(deque)
suspicious = {}

pattern = re.compile(r"^(\d\d):(\d\d)\s+(\S+)\s+(\S+)\s+(FAIL|SUCCESS)$")

for _ in range(n):
    line = input().strip()
    match = pattern.match(line)

    if not match:
        continue

    hour = int(match.group(1))
    minute = int(match.group(2))
    user = match.group(3)
    ip = match.group(4)
    status = match.group(5)

    current = hour * 60 + minute

    if status == "FAIL":
        failed[user].append((current, ip))
        while failed[user] and current - failed[user][0][0] > t:
            failed[user].popleft()

    else:
        while failed[user] and current - failed[user][0][0] > t:
            failed[user].popleft()

        if len(failed[user]) >= x:
            different = False
            for _, old_ip in failed[user]:
                if old_ip != ip:
                    different = True
                    break

            if different and user not in suspicious:
                suspicious[user] = f"{hour:02d}:{minute:02d}"

for user in sorted(suspicious):
    print(user, suspicious[user])
```

Output

```
Program Output

ul 00:07
```

PROGRAM 7

Program Code

```
import json

def read_records(path):
    with open(path, "r", encoding="utf-8") as file:
        for line in file:
            try:
                data = json.loads(line)

                if "device_id" not in data:
                    continue
                if "temperature_c" not in data:
                    continue
                if "humidity" not in data:
                    continue

                temperature = float(data["temperature_c"])
                humidity = float(data["humidity"])

                yield data["device_id"], temperature, humidity
            except:
                yield None, None, None

path = input("Enter JSONL file path: ").strip()

stats = {}
corrupted = {}

try:
    with open(path, "r", encoding="utf-8") as file:
        for line in file:
            try:
                data = json.loads(line)

                if "device_id" not in data or "temperature_c" not in data or "humidity"
                    not in data:
                    raise ValueError

                device = data["device_id"]
                temperature = float(data["temperature_c"])
                float(data["humidity"])

                if device not in stats:
                    stats[device] = [0, temperature, temperature, 0]

                stats[device][0] += 1
                stats[device][1] = min(stats[device][1], temperature)
                stats[device][2] = max(stats[device][2], temperature)
                stats[device][3] += temperature

            except:
                device = "UNKNOWN"
                corrupted[device] = corrupted.get(device, 0) + 1

    for device in sorted(stats):
        count, minimum, maximum, total = stats[device]
        bad = corrupted.get(device, 0)
        average = total / count
        print(
            device,
            "count=" + str(count),
            "min=" + str(minimum),
            "max=" + str(maximum),
            "avg=" + f"{average:.2f}",
            "corrupted=" + str(bad)
        )

except FileNotFoundError:
    print("File not found")
```

Output

```
Program Output

Enter JSONL file path: D1 count=1 min=30.0 max=30.0 avg=30.00 corrupted=0
```

PROGRAM 8

Program Code

```
MOD = 1000000007

n, m = map(int, input().split())
matrix = []

for _ in range(n):
    matrix.append(input().split())

score = [[None] * m for _ in range(n)]
ways = [[0] * m for _ in range(n)]

if matrix[0][0] != "X":
    score[0][0] = int(matrix[0][0])
    ways[0][0] = 1

for i in range(n):
    for j in range(m):
        if matrix[i][j] == "X":
            continue

        if i == 0 and j == 0:
            continue

        value = int(matrix[i][j])
        best = None
        count = 0

        previous = []

        if i > 0:
            previous.append((score[i - 1][j], ways[i - 1][j]))

        if j > 0:
            previous.append((score[i][j - 1], ways[i][j - 1]))

        if i > 0 and j > 0:
            previous.append((score[i - 1][j - 1], ways[i - 1][j - 1]))

        for old_score, old_count in previous:
            if old_score is None or old_count == 0:
                continue

            if best is None or old_score > best:
                best = old_score
                count = old_count
            elif old_score == best:
                count = (count + old_count) % MOD

        if best is not None:
            score[i][j] = best + value
            ways[i][j] = count

if score[n - 1][m - 1] is None:
    print("IMPOSSIBLE")
else:
    print(score[n - 1][m - 1], ways[n - 1][m - 1])
```

Output

```
Program Output

291
```

PROGRAM 9

Program Code

```
class Product:
    def __init__(self, pid, name, stock, purchase, selling):
        self.pid = pid
        self.name = name
        self.stock = stock
        self.purchase = purchase
        self.selling = selling

class Inventory:
    def __init__(self):
        self.products = {}

    def add(self, product):
        self.products[product.pid] = product

    def delete(self, pid):
        if pid in self.products:
            del self.products[pid]
            return True
        return False

    def update(self, pid, stock):
        if pid in self.products:
            self.products[pid].stock = stock
            return True
        return False

    def total_value(self):
        total = 0
        for product in self.products.values():
            total += product.stock * product.purchase
        return total

    def __lt__(self, other):
        return self.total_value() < other.total_value()

    def __add__(self, other):
        result = Inventory()

        for pid, product in self.products.items():
            result.add(Product(
                pid,
                product.name,
                product.stock,
                product.purchase,
                product.selling
            ))

        for pid, product in other.products.items():
            if pid not in result.products:
                result.add(Product(
                    pid,
                    product.name,
                    product.stock,
                    product.purchase,
                    product.selling
                ))
            else:
                old = result.products[pid]
                old.stock += product.stock
                old.purchase = min(old.purchase, product.purchase)
                old.selling = max(old.selling, product.selling)

        return result

def read_inventory():
    inventory = Inventory()
    n = int(input())
    for _ in range(n):
        pid, name, stock, purchase, selling = input().split()
        inventory.add(Product(
            pid,
            name,
            int(stock),
            float(purchase),
            float(selling)
        ))
    return inventory

print("Enter Inventory A")
a = read_inventory()

print("Enter Inventory B")
b = read_inventory()

operation = input("Operation: ").strip().upper()

if operation == "MERGE":
```

```
    result = a + b
    for pid in sorted(result.products):
        p = result.products[pid]
        print(
            p.pid,
            p.name,
            "stock=" + str(p.stock),
            "purchase=" + str(int(p.purchase) if p.purchase.is_integer() else p.purchase),
            "selling=" + str(int(p.selling) if p.selling.is_integer() else p.selling)
        )
    print("TOTAL", result.total_value())

elif operation == "COMPARE":
    if a < b:
        print("Inventory A has lower value")
    elif b < a:
        print("Inventory B has lower value")
    else:
        print("Both inventories have equal value")
else:
    print("Invalid operation")
```

Output

```
Program Output

Enter Inventory A
Enter Inventory B

Operation: P1 Pen stock=17 purchase=4 selling=9

TOTAL 68.0
```

PROGRAM 10

Program Code

```
import csv
import os
import matplotlib.pyplot as plt

input_file = input("Enter marks CSV path: ").strip()
output_folder = input("Enter output folder: ").strip()

os.makedirs(output_folder, exist_ok=True)

try:
    with open(input_file, "r", encoding="utf-8", newline="") as file:
        rows = list(csv.DictReader(file))

    if not rows:
        print("No data")
        raise SystemExit

    fields = list(rows[0].keys())
    subjects = fields[2:]

    for row in rows:
        for subject in subjects:
            value = row[subject].strip()
            if value == "":
                row[subject] = "0"
            else:
                mark = float(value)
                if mark < 0 or mark > 100:
                    row[subject] = "0"

    cleaned_file = os.path.join(output_folder, "cleaned_marks.csv")
    summary_file = os.path.join(output_folder, "summary.csv")

    with open(cleaned_file, "w", encoding="utf-8", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)

    averages = []
    for subject in subjects:
        total = 0
        for row in rows:
            total += float(row[subject])
        averages.append(total / len(rows))

    with open(summary_file, "w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["Subject", "Average"])
        for subject, average in zip(subjects, averages):
            writer.writerow([subject, f"{average:.2f}"])

    grade_count = {"A": 0, "B": 0, "C": 0, "D": 0, "F": 0}

    for row in rows:
        total = 0
        for subject in subjects:
            total += float(row[subject])
        average = total / len(subjects)

        if average >= 80:
            grade_count["A"] += 1
        elif average >= 70:
            grade_count["B"] += 1
        elif average >= 60:
            grade_count["C"] += 1
        elif average >= 50:
            grade_count["D"] += 1
        else:
            grade_count["F"] += 1

    plt.figure()
    plt.bar(grade_count.keys(), grade_count.values())
    plt.title("Grade Distribution")
    plt.xlabel("Grade")
    plt.ylabel("Students")
    plt.savefig(os.path.join(output_folder, "grade_distribution.png"))
    plt.close()

    plt.figure()
    plt.bar(subjects, averages)
    plt.title("Subject Average")
    plt.xlabel("Subject")
    plt.ylabel("Average Marks")
    plt.savefig(os.path.join(output_folder, "subject_average.png"))
    plt.close()

    student_scores = []
    for row in rows:
```

```
            total = 0
            for subject in subjects:
                total += float(row[subject])
            student_scores.append((row["name"], total / len(subjects)))

        student_scores.sort(key=lambda x: x[1], reverse=True)
        top = student_scores[:5]

        names = [x[0] for x in top]
        scores = [x[1] for x in top]

        plt.figure()
        plt.bar(names, scores)
        plt.title("Top Performers")
        plt.xlabel("Student")
        plt.ylabel("Average Marks")
        plt.xticks(rotation=30)
        plt.savefig(os.path.join(output_folder, "top_performers.png"))
        plt.close()

        print("cleaned_marks.csv")
        print("summary.csv")
        print("grade_distribution.png")
        print("subject_average.png")
        print("top_performers.png")

    except FileNotFoundError:
        print("File not found")
```

Output

```
Program Output

Enter marks CSV path: Enter output folder: cleaned_marks.csv

summary.csv

grade_distribution.png

subject_average.png

top_performers.png
```

The program opens a graphical application window. The screenshot above shows the interface and sample records.